

Messaging System in Microservice architecture

There are two primary types of messaging systems: **Message-Oriented Middleware (MOM)** and **Event-Driven Architecture (EDA)**.

Message-Oriented Middleware (MOM):

Message-oriented middleware revolves around the concept of messages being sent and received between distributed systems. This type of messaging system is characterized by **asynchronous communication**, where the sender and receiver are **decoupled**. MOM typically involves the use of **message queues** for storing and managing messages until they are consumed by the intended recipient.

What are message queues?

One of the key challenges of microservices architecture is managing communication between these services in a scalable and efficient manner. This is where message queues come in, allowing the microservices to communicate asynchronously and reliably.

A message queue, as the name says, is a queue that routes messages from the sender to the receiver, (from service A to service B) following the **FIFO** (First In, First Out) policy. This approach decouples the sender and receiver, allowing each service to operate independently without being tightly coupled to the others. However, you can have **multiple consumers (instances) connected to the same queue**.

Why/when do we need them?

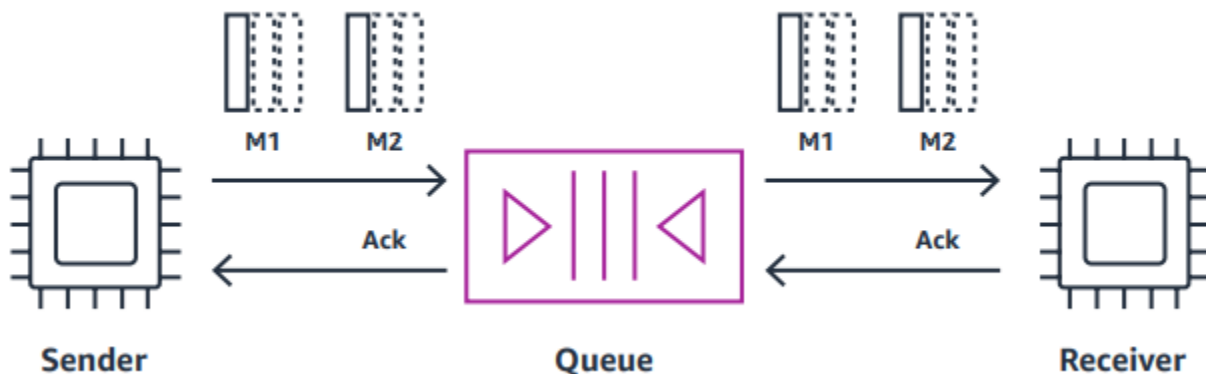
Message queues bring some great benefits to the table and also have some trade-offs, which you should take into account when deciding what type of communication/model you need/want in your specific use case.

1. **Asynchronous communication**: Rather than waiting for a response before processing the next request, a service can send a message to a queue and move on to the next request, allowing the system to process more requests in **parallel**.
2. **Scalability**: Because messages are stored in a queue, **multiple instances** of a service can process messages from the same queue, enabling the system to handle more traffic as demand increases.
3. **Reliability**: Message queues can help improve reliability by guaranteeing message delivery. **There are 4 types of message delivery: at most once, at least once, exactly once & best-effort message delivery.**

4. **Loose coupling:** Services can be written in **different programming languages** or run on different platforms as long as they can both read and write messages to the same queue. (Generally, the payload of a message is in JSON format)
5. **Buffering:** Messages can be stored in a queue when the receiving service is **not** immediately **available** to process them. (helping with the reduction of load on the services and prevention of overload)
6. **Message transformations:** Messages can be transformed or modified before they are delivered to the receiving service.
7. **Prioritization**, meaning that based on a set **priority**, high-priority messages can be processed first and low-priority messages can even be throttled to prevent overloading of services.

Point-to-point (P2P)

In this model a message published by the producer is received only by one consumer, hence the name point-to-point. Point-to-point messaging often uses messaging queues. In microservices applications, asynchronous, point-to-point messaging between microservices is referred to as "**dumb pipes**".



Queues are messaging channels that allow asynchronous communication between a sender and receiver. **Queues provide a buffer for messages in case the consumer is unavailable or needs to control the number of messages processed at a given time.** Messages persist until the consumer processes them and deletes them from the queue.”

Examples: Services like Amazon Simple Queue Service (Amazon SQS) and Amazon MQ are commonly used as message queues in event-driven architectures. You can also use asynchronous AWS Lambda invocations.

The advantages of using this model are:

- High throughput and low latency, making it suitable for real-time applications.

- Provides reliable message delivery, since each message is delivered to a specific destination and consumed by a single consumer.
- Enables parallel processing of messages, which can be processed concurrently by different consumers
- Ensures that each message is processed only once, reducing the risk of duplicate messages.

And the disadvantages are:

- Limited flexibility
- Not suitable for scenarios where messages need to be broadcast to multiple consumers simultaneously.
- Can be more complex to implement than other message queue models.

Publish/Subscribe (Pub/Sub):

A communications paradigm known as the Publisher/Subscriber pattern involves senders (publishers) broadcasting messages without knowing who the recipients are (subscribers). Without knowing the publishers, subscribers indicate interest in one or more themes, and they get interesting communications. The system's flexibility and scalability are improved by this separation of publishers and subscribers. Subscribers express interest in certain types of messages, and they receive relevant messages when published.

Key Concepts:

Publisher: The component that sends messages.

Subscriber: The component that receives messages.

Message: The data being transmitted.

Channel/Topic: The medium through which messages are sent.

Exchange:

An exchange in messaging systems (like RabbitMQ) is the **routing mechanism** that decides how messages are delivered from a producer to one or more queues.

Producers never send messages directly to queues; they send them to an **exchange**, and the exchange routes them based on rules.

Producer → Exchange → Queue(s) → Consumer(s)

The exchange takes a message and decides:

- Which queue(s) should get it?
- Should it go to one queue, many queues, or none?

Types of Exchanges in RabbitMQ

1. Direct Exchange

- Routes messages to queues with an exact matching routing key.
- Think of it like a mailbox with a label: only the exact label gets the mail.

Example:

- Routing keys: email, sms, push
- Queue A is bound with email, Queue B with sms.
- Message with key email → goes to Queue A only.

Use case: Task routing where each task type has a dedicated worker.

2. Fanout Exchange

- Sends messages to all queues bound to it, ignoring the routing key.
- Think of it like a radio broadcast—everyone tuned in hears it.

Example:

- Exchange has 3 queues bound: audit_log, notification_service, analytics.
- Producer sends one message → all three queues get a copy.

Use case: Pub/Sub scenarios—notifications, system-wide updates.

3. Topic Exchange

- Routes messages based on pattern matching with routing keys (wildcards).
- Routing keys are usually dot-separated words, e.g., order.created.us.

Example:

- Queue A binds with order.* (matches order.created or order.cancelled).
- Queue B binds with order.*.us (matches only US orders).
- Message with key order.created.us → goes to both A and B.

Use case: Flexible routing for event-driven systems.

4. Headers Exchange

- Uses headers (key-value pairs) in the message instead of routing keys.
- Matches based on headers set by producer.

Example:

- Queue binds with headers {type: "invoice", format: "pdf"}.
- Only messages with matching headers go to this queue.

Use case: When routing logic is based on metadata, not just type.

The advantages of using this model are:

- Offers more flexibility than the p2p model, giving you the chance for a many-to-many relationship.
- Enables dynamic discovery of consumers.
- The producer of a message does not need to know who the consumers are.

And the disadvantages are:

- Can lead to higher network traffic and overhead, especially when multiple subscribers are involved.
- It's more complex to implement.
- May need more measures to make sure that each subscriber consistently receives the same message.

Code Example:

Pure P2P (Point-to-Point) Version

In P2P:

- **One queue → One consumer processes each message.**
- **Multiple consumers can listen, but each message is delivered to only one of them.**

```
const amqp = require('amqplib/callback_api');

class PointToPoint {
  constructor() {
    this.connection = null;
    this.channel = null;
  }

  async connect() {
    return new Promise((resolve, reject) => {
      amqp.connect('amqp://localhost', (error, connection) => {
        if (error) return reject(error);
        this.connection = connection;
        connection.createChannel((error, channel) => {
          if (error) return reject(error);
          this.channel = channel;
        });
      });
    });
  }
}
```

```

        resolve();
      });
    });
  }

  send(queue, message) {
    this.channel.assertQueue(queue, { durable: true }); // Durable for reliability
    this.channel.sendToQueue(queue, Buffer.from(message), { persistent: true }); //
    Persistent message
    console.log(`Sent: ${message}`);
  }

  receive(queue, callback) {
    this.channel.assertQueue(queue, { durable: true });
    this.channel.consume(queue, (msg) => {
      callback(msg.content.toString());
      this.channel.ack(msg); // Acknowledge message
    });
  }
}

// Usage
(async () => {
  const p2p = new PointToPoint();
  await p2p.connect();

  // Consumer
  p2p.receive('taskQueue', (message) => {
    console.log(`Processing: ${message}`);
  });

  // Producer
  p2p.send('taskQueue', JSON.stringify({ task: 'Email user', userId: 42 }));
})();

```

Key P2P points:

- Messages go to only one consumer per message.
- `ack()` ensures no message is lost if a consumer crashes.
- Multiple consumers = load balancing.

Convert to True Pub/Sub

In Pub/Sub:

- **Publisher → Exchange → Multiple queues.**
- Each subscriber gets its **own copy of the message.**

```
const amqp = require('amqplib/callback_api');

class PubSub {
  constructor() {
    this.connection = null;
    this.channel = null;
    this.exchange = 'broadcastExchange';
  }

  async connect() {
    return new Promise((resolve, reject) => {
      amqp.connect('amqp://localhost', (error, connection) => {
        if (error) return reject(error);
        this.connection = connection;
        connection.createChannel((error, channel) => {
          if (error) return reject(error);
          this.channel = channel;
          this.channel.assertExchange(this.exchange, 'fanout', { durable: true });
          resolve();
        });
      });
    });
  }

  publish(message) {
    this.channel.publish(this.exchange, "", Buffer.from(message));
    console.log(`Published: ${message}`);
  }

  subscribe(callback) {
    this.channel.assertQueue("", { exclusive: true }, (error, q) => {
      if (error) throw error;

      this.channel.bindQueue(q.queue, this.exchange, "");
      this.channel.consume(q.queue, (msg) => {
        callback(msg.content.toString());
      }, { noAck: true });
    });
  }
}
```

```
}

// Usage
(async () => {
  const pubsub = new PubSub();
  await pubsub.connect();

  // Subscriber 1
  pubsub.subscribe((message) => {
    console.log(`Subscriber 1 got: ${message}`);
  });

  // Subscriber 2
  pubsub.subscribe((message) => {
    console.log(`Subscriber 2 got: ${message}`);
  });

  // Publisher
  pubsub.publish(JSON.stringify({ event: 'UserSignedUp', user: 'John Doe' }));
})();
```

Key Pub/Sub points:

- Each subscriber gets its own queue.
- fanout exchange broadcasts to all queues bound to it.
- **bindQueue** creates a **binding** between a **queue** and an **exchange**, using a **routing key** (depending on the exchange type).
- No message acknowledgment needed if real-time events are acceptable.